ALGORITHM SPECIFICATION & ANALYSIS REPORT

Max-Flow Min-Cut & Cut Set Identification

Residual capacity augmentation and exact edge identification for network minimum cut sets

1. Algorithm Code (Part I: DFS Traversal Method)

Below is Part 1 of the implementation, showing the recursive depth-first search function used to discover augmenting paths and trace predecessor links in the residual graph:

```
import java.util.*;

public class EdmondsKarp {

    static boolean dfs(int[][] residual, int u, int sink,
                       boolean[] visited, int[] parent) {

        visited[u] = true;

        if (u == sink) {
            return true;
        }

        for (int v = 0; v < residual.length; v++) {
            if (!visited[v] && residual[u][v] > 0) {
                parent[v] = u;

                if (dfs(residual, v, sink, visited, parent)) {
                    return true;
                }
            }
        }

        return false;
    }
```

Augmenting Path Identification: Path reachability is verified recursively. When the sink node is reached, the recursion returns `true` and the augmenting path is reconstructed via `parent[]`.

1. Algorithm Code (Part II: Max Flow & Cut Set Extraction)

Below is Part 2 of the implementation, calculating max flow, capturing the final reachable partition, and printing the active cut set edges:

```
    static int maxFlow(int[][] capacity, int source, int sink) {
        int n = capacity.length;
        int[][] residual = new int[n][n];

        for (int i = 0; i < n; i++) {
            residual[i] = capacity[i].clone();
        }

        int[] parent = new int[n];
        int maxFlow = 0;
        boolean[] finalVisited = new boolean[n];

        while (true) {
            boolean[] visited = new boolean[n];
            Arrays.fill(parent, -1);

            if (!dfs(residual, source, sink, visited, parent)) {
                finalVisited = visited;
                break;
            }

            int pathFlow = Integer.MAX_VALUE;

            for (int v = sink; v != source; v = parent[v]) {
                int u = parent[v];
                pathFlow = Math.min(pathFlow, residual[u][v]);
            }

            for (int v = sink; v != source; v = parent[v]) {
                int u = parent[v];
                residual[u][v] -= pathFlow;
                residual[v][u] += pathFlow;
            }

            maxFlow += pathFlow;
        }

        System.out.println("Capacity of the Minimum Cut (Max Flow): " + maxFlow);
        System.out.println("Edges in the Minimum Cut set:");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (finalVisited[i] && !finalVisited[j] && capacity[i][j] > 0) {
                    System.out.println(i + " -> " + j + " (Capacity: " + capacity[i][j] + ")")
                }
            }
        }

        return maxFlow;
    }

    public static void main(String[] args) {
        int[][] capacity = {
            {0, 10, 5, 0},
            {0, 0, 5, 0}
```

2. Output & Output Explanation

```
Capacity of the Minimum Cut (Max Flow): 10 Edges in the Minimum Cut set: 0 -> 2
(Capacity: 5) 1 -> 2 (Capacity: 5)
```

The algorithm computes the maximum flow and isolates the cut set edges between source Node 0 and sink Node 2:

- **Path 1 Augmentation (0 → 1 → 2):** Bottleneck capacity is `min(capacity[0][1], capacity[1][2]) = min(10, 5) = 5`. Residuals update: `residual[0][1] = 5`, `residual[1][2] = 0`. Current flow becomes **5**.
- **Path 2 Augmentation (0 → 2):** Discovers direct path 0 → 2 with bottleneck capacity 5. Forward residual becomes 0. Total flow increases to 5 + 5 = **10**.
- **Termination & Reachability:** A subsequent DFS traversal from Node 0 can reach Node 1 (residual 5), but cannot reach Node 2 (residual 0) or Node 3. Hence, the final reachable set is **S = {0, 1}** and the unreachable set is **T = {2, 3}**.
- **Cut Capacity:** Edges crossing from S to T are 0 → 2 (capacity 5) and 1 → 2 (capacity 5). Total capacity = 5 + 5 = **10**, matching the max flow exactly.

3. Code Walkthrough

Component / Variable	Mechanics & Implementation Purpose
`finalVisited` array	Preserves the visited array of the final failed DFS iteration, defining the source component S of the cut.
`dfs(...)` routine	Traverses edges with `residual[u][v] > 0` to locate path augmentations between source and sink.
`parent[]` tracking	Stores predecessor nodes along the path to calculate bottleneck flow and perform residual edge updates.
Residual adjustments	Decrements forward edge capacities and increments backward edge capacities by `pathFlow`.
Cut edge identification	Scans all pairs (i, j); if $i \in S$ and $j \in T$ with positive capacity, the edge is part of the minimum cut.

4. Algorithmic Foundation & Working Principle

This implementation validates the fundamental theorems of maximum network flow:

- **Max-Flow Min-Cut Theorem:** In any flow network, the maximum flow value is equal to the capacity of the minimum cut separating source and sink.
- **Linear Programming Duality:** Max-flow is the primal problem and Min-cut is the dual problem. The zero duality gap guarantees equality at the optimal solution.

MAX FLOW / MIN CUT

10 Units

Edges: 0→2 (5) + 1→2 (5) = 10.

CUT PARTITION (S, T)

{0, 1} | {2, 3}

Reachable vs unreachable vertices.

CUT EXTRACTION

$\Theta(V^2)$

Single pass over capacity matrix.

5. Practical Uses & Algorithmic Scope

Extracting explicit bottleneck cut edges enables critical infrastructure optimizations:

- **Vulnerability & Bottleneck Analysis:** Directly identifies the critical physical links that limit overall network throughput.
- **Network Bisection & Partitioning:** Evaluates minimum link failures needed to split communication infrastructure.
- **Project Selection Optimization:** Identifies optimal configurations of dependent projects and revenue streams via closed sets.

6. Real-World Industry Use Cases

Industry Domain	Specific Scenario	Min-Cut Application
Medical Imaging (MRI / CT Scans)	Organ and tumor segmentation.	Formulates pixel graphs where edge costs reflect intensity gradients; min-cut isolates boundaries with minimal energy.
Power Grid Reliability	Blackout prevention in transmission lines.	Pinpoints vulnerable high-voltage transmission lines under heavy load to prevent cascading power failures.
Cloud Infrastructure	Subnet DDoS mitigation and traffic isolation.	Isolates compromised subnet partitions from operational services with minimal disruption to adjacent workloads.
Utility Infrastructure	Targeted municipal pipeline upgrades.	Guides municipal investment toward widening the specific physical pipes present in the cut set.